



Documentación Base de Domain-Dri

CampusMarket

1. Descripción General del Proyecto

1.1 Nombre del Proyecto

CampusMarket

1.2 Descripción del Proyecto

CampusMarket es una plataforma de marketplace universitario diseñada para estudiantes de la Universidad Central.

La plataforma permite a los estudiantes:

- Comprar y vender productos académicos
- Ofrecer servicios de tutoría
- Publicar servicios freelance
- Construir reputación académica
- Interactuar dentro de un ecosistema universitario confiable

El sistema se enfoca en:

- Autenticación segura
- Confianza académica
- Interacciones del marketplace
- Gestión de tutorías
- Seguimiento de reputación

2. Estilo Arquitectónico

2.1 Arquitectura Seleccionada

El sistema utiliza:

- Arquitectura Monolítica Modular
- Arquitectura por Capas
- Domain-Driven Design
- Arquitectura API REST

2.2 ¿Por qué Monolito Modular?

El proyecto está pensado para ser desarrollado por un pequeño equipo académico en un corto período de tiempo.

Un monolito modular proporciona:

- Menor complejidad operativa

- Despliegue más sencillo
- Desarrollo más rápido
- Separación clara del dominio
- Depuración más sencilla
- Mejor mantenibilidad

Comparada con los microservicios, esta arquitectura evita:

- Complejidad de sistemas distribuidos
- Descubrimiento de servicios
- Sobrecarga de comunicación en red
- Despliegues independientes
- Complejidad de orquestación de contenedores

2.3 Capas Arquitectónicas

La aplicación se divide en cuatro capas:

Capa de Dominio

Contiene:

- Entidades
- Objetos de Valor
- Agregados
- Servicios de Dominio
- Contratos de Repositorio
- Reglas de Negocio

Esta capa no depende del Framework Spring.

Capa de Aplicación

Contiene:

- Casos de Uso
- Servicios de Aplicación
- Lógica de Orquestación
- Coordinación de Transacciones

Capa de Infraestructura

Contiene:

- Repositorios JPA
- Integración con PostgreSQL
- Autenticación JWT
- Spring Security
- APIs Externas
- Servicios en la Nube

Capa de Interfaz

Contiene:

- Controladores REST

- Solicitudes HTTP
- Respuestas HTTP
- Endpoints API

3. Diseño Estratégico

3.1 Dominio Principal

El Dominio Principal de CampusMarket es:

Interacción de Marketplace Académico

Esto incluye:

- Publicaciones de productos estudiantiles
- Oferta de servicios
- Interacciones de tutorías
- Reputación académica

Este dominio representa el principal valor de negocio del sistema.

3.2 Subdominios de Soporte

Gestión de Identidad

Responsable de autenticación y autorización.

Sistema de Reputación

Responsable del cálculo de confianza y reseñas.

4. Contextos Delimitados

El sistema está dividido en contextos delimitados.

Cada contexto delimitado:

- Posee sus propias reglas de negocio
- Define su propio lenguaje
- Encapsula sus propios modelos
- Reduce el acoplamiento

5. Contexto de Identidad

5.1 Propósito

Responsable de:

- Registro de usuarios
- Autenticación
- Autorización
- Gestión JWT
- Verificación institucional

5.2 Lenguaje Ubicuo

Término	Definición
Usuario	Estudiante autenticado
Usuario Verificado	Usuario validado institucionalmente
Puntaje de Confianza	Indicador de confianza
Correo Institucional	Correo universitario
Autenticación	Proceso de inicio de sesión
Autorización	Validación de acceso
Token JWT	Token de autenticación

5.3 Raíz del Agregado

Usuario

El agregado Usuario controla:

- Consistencia de identidad
- Reglas de autenticación
- Validación de seguridad
- Gestión de roles

5.4 Entidades

Usuario

Representa un miembro autenticado de la plataforma.

Atributos:

- id
- fullName
- email
- password
- role
- verified
- trustScore
- createdAt

Responsabilidades:

- Mantener el estado de la cuenta
- Validar reglas de identidad
- Controlar el acceso de autenticación

5.5 Objetos de Valor

Email

Representa un correo institucional válido.

Reglas:

- No puede ser nulo
- Debe tener formato válido
- Debe pertenecer a dominios aceptados

Contraseña

Representa credenciales cifradas.

Reglas:

- Debe estar cifrada
- No puede exponer el valor en texto plano

5.6 Repositorios

UserRepository

Responsabilidades:

- Persistir usuarios
- Buscar usuarios por correo
- Validar unicidad

5.7 Servicios de Dominio

AuthenticationService

Responsable de:

- Validación de inicio de sesión
- Verificación de credenciales
- Reglas de autenticación

TrustVerificationService

Responsable de:

- Validación institucional
- Actualización del puntaje de confianza

5.8 Invariantes

El agregado Usuario garantiza:

- Correo único
- Contraseña cifrada
- Puntaje de confianza válido
- Rol válido

6. Contexto Marketplace

6.1 Propósito

Responsable de:

- Publicaciones de productos
- Publicaciones de servicios
- Gestión de publicaciones
- Interacciones del marketplace

Este es el Dominio Principal.

6.2 Lenguaje Ubicuo

Término	Definición
Publicación	Publicación del marketplace
Publicación de Producto	Publicación de producto
Publicación de Servicio	Publicación de servicio
Vendedor	Usuario que publica
Borrador	Publicación incompleta
Publicación Publicada	Publicación visible
Categoría	Clasificación de publicación
Precio	Valor monetario

6.3 Raíz del Agregado

Listing

Controla:

- Ciclo de vida de la publicación
- Consistencia de publicaciones
- Reglas de negocio

6.4 Entidades

Listing

Representa una publicación del marketplace.

Atributos:

- id
- title
- description
- price

- category
- ownerId
- status
- createdAt

Responsabilidades:

- Mantener estado de la publicación
- Controlar ciclo de vida
- Validar reglas de publicación

Category

Representa una clasificación de publicación.

Ejemplos:

- Libros
- Electrónica
- Tutorías
- Servicios de Diseño
- Comida

ListingImage

Representa imágenes de publicaciones.

Responsabilidades:

- Asociar multimedia a publicaciones
- Almacenar referencias de imágenes

6.5 Objetos de Valor

Precio

Representa valor monetario.

Reglas:

- Debe ser positivo
- No puede ser negativo

Título de Publicación

Representa el título de una publicación.

Reglas:

- No puede estar vacío
- Debe respetar límites de longitud

Descripción de Publicación

Representa detalles de publicación.

Reglas:

- No puede estar vacía

- Debe contener contenido válido

6.6 Repositorios

ListingRepository

Responsabilidades:

- Persistir publicaciones
- Buscar publicaciones
- Filtrar publicaciones

CategoryRepository

Responsabilidades:

- Gestionar categorías
- Validar existencia de categorías

6.7 Servicios de Dominio

RecommendationService

Responsable de:

- Recomendaciones de publicaciones
- Sugerencias personalizadas

ContentModerationService

Responsable de:

- Detectar contenido inválido
- Prevenir publicaciones abusivas
- Filtrar publicaciones prohibidas

6.8 Invariantes

El agregado Listing garantiza:

- La publicación debe tener propietario
- La publicación debe tener categoría
- El precio debe ser positivo
- Las publicaciones publicadas deben estar completas

7. Contexto de Tutorías

7.1 Propósito

Responsable de:

- Servicios de tutoría
- Programación de sesiones
- Disponibilidad de tutores

- Asistencia académica

7.2 Lenguaje Ubicuo

Término	Definición
Tutor	Estudiante que ofrece tutorías
Sesión	Cita de tutoría
Materia	Asignatura académica
Disponibilidad	Horario del tutor
Tarifa por Hora	Precio de sesión

7.3 Raíz del Agregado

TutorProfile

Controla:

- Información del tutor
- Consistencia de disponibilidad
- Gestión de materias

7.4 Entidades

TutorProfile

Representa proveedor de tutorías.

Atributos:

- id
- userId
- subjects
- hourlyRate
- availability
- biography

TutoringSession

Representa tutoría reservada.

Atributos:

- id
- tutorId
- studentId
- subject
- scheduledAt
- status

7.5 Objetos de Valor

Schedule

Representa disponibilidad de tutorías.

HourlyRate

Representa precio de tutorías.

Reglas:

- Debe ser positivo

7.6 Repositorios

TutorProfileRepository

TutoringSessionRepository

7.7 Servicios de Dominio

TutorMatchingService

Responsable de:

- Encontrar tutores
- Relacionar materias
- Sugerir tutores disponibles

AvailabilityService

Responsable de:

- Validar horarios
- Prevenir solapamientos

8. Contexto de Reputación

8.1 Propósito

Responsable de:

- Gestión de reputación
- Reseñas
- Métricas de confianza
- Detección de fraude

8.2 Lenguaje Ubicuo

Término	Definición
Reseña	Retroalimentación de usuario
Calificación	Evaluación numérica

Reputación	Credibilidad del usuario
Fraude	Comportamiento sospechoso
Puntaje de Reputación	Indicador de confianza

8.3 Raíz del Agregado

ReputationScore

Controla:

- Consistencia de reputación
- Cálculos de calificaciones

8.4 Entidades

Review

Representa retroalimentación del usuario.

Atributos:

- id
- reviewerId
- targetUserId
- rating
- comment
- createdAt

ReputationScore

Representa reputación acumulada.

Atributos:

- userId
- averageScore
- totalReviews

8.5 Objetos de Valor

Rating

Representa puntuación numérica.

Reglas:

- Valor mínimo: 1
- Valor máximo: 5

Comments

Representa mensaje de reseña.

Reglas:

- No puede exceder límite de tamaño

8.6 Repositorios

ReviewRepository

Responsabilidades:

- Persistir reseñas
- Buscar reseñas

8.7 Servicios de Dominio

ReputationCalculationService

Responsable de:

- Calcular promedios
- Actualizar métricas de confianza

FraudDetectionService

Responsable de:

- Detectar comportamiento sospechoso
- Prevenir reseñas falsas

9. Mapa de Contextos

Los contextos delimitados interactúan mediante relaciones controladas.

9.1 Identity → Marketplace

Marketplace requiere usuarios autenticados.

Comunicación:

- ID de usuario
- Rol de usuario

Identity no expone lógica interna de autenticación.

9.2 Identity → Tutoring

Tutoring requiere usuarios verificados.

Comunicación:

- Identidad de usuario
- Estado de verificación

9.3 Marketplace → Reputation

Marketplace envía interacciones completadas.

Reputation calcula:

- Confianza del vendedor
- Métricas de reseñas

9.4 Tutoring → Reputation

Las sesiones de tutoría generan:

- Reseñas de tutores
- Retroalimentación de estudiantes

10. Eventos de Dominio

El sistema utiliza eventos de dominio para representar ocurrencias importantes del negocio.

10.1 UserRegisteredEvent

Se activa cuando un usuario se registra.

10.2 ListingPublishedEvent

Se activa cuando una publicación se vuelve visible.

10.3 TutoringSessionCompletedEvent

Se activa después de completar una tutoría.

10.4 ReviewCreatedEvent

Se activa cuando una reseña es enviada.

11. Estrategia de Persistencia

Base de Datos

PostgreSQL

Hospedada en:

Supabase

ORM

Spring Data JPA + Hibernate

Estrategia de Identificadores

Se utilizan identificadores UUID para todos los agregados.

Ventajas:

- Unicidad distribuida
- Seguridad
- Mejor escalabilidad

12. Estrategia de Seguridad

Autenticación

Autenticación JWT

Autorización

Autorización basada en roles.

Roles:

- STUDENT
- ADMIN

Cifrado de Contraseñas

BCrypt Password Encoder

13. Estrategia de Despliegue

Backend

Aplicación Spring Boot

Base de Datos

Supabase PostgreSQL

Opciones Futuras de Despliegue

- AWS EC2
- Railway
- Render
- Despliegue con Docker

14. Estructura del Proyecto

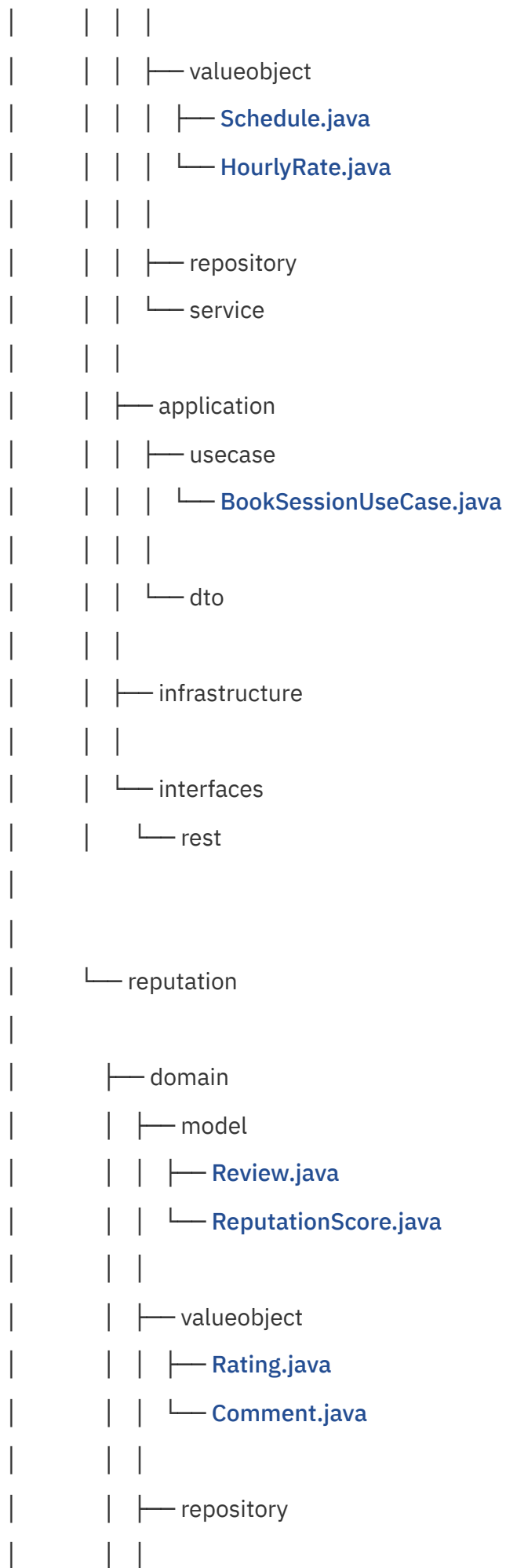
src

```
└─ main
    │   └─ java
    │       └─ com
    │           └─ campusmarket
    │               └─ CampusMarketApplication.java
    │
    │   └─ shared
    │       └─ exception
    │       └─ config
    │       └─ util
```

```
| | └─ response
| |
| | └─ identity
| |
| | └─ domain
| |   └─ model
| |     └─ User.java
| |     └─ Role.java
| |       └─ UserStatus.java
| |
| |   └─ valueobject
| |     └─ Email.java
| |     └─ Password.java
| |
| |   └─ repository
| |     └─ UserRepository.java
| |
| |   └─ service
| |     └─ AuthenticationService.java
| |     └─ TrustVerificationService.java
| |
| |   └─ exception
| |
| | └─ application
| |   └─ usecase
| |     └─ RegisterUserUseCase.java
| |     └─ LoginUserUseCase.java
| |
| |   └─ dto
| |     └─ RegisterRequest.java
| |     └─ LoginRequest.java
| |     └─ AuthResponse.java
| |
```

```
|      | |  └─ service
|      | |
|      | └─ infrastructure
|      | |  └─ persistence
|      | | |  └─ UserJpaEntity.java
|      | | |  └─ JpaUserRepository.java
|      | | |  └─ UserRepositoryAdapter.java
|      | | |
|      | | └─ security
|      | | |  └─ JwtService.java
|      | | |  └─ JwtFilter.java
|      | | |  └─ CustomUserDetailsService.java
|      | | |  └─ SecurityConfiguration.java
|      | | |
|      | | └─ mapper
|      | |
|      | └─ interfaces
|      |   └─ rest
|      |     └─ AuthenticationController.java
|
|
|
|      └─ marketplace
|
|
|      └─ domain
|      |  └─ model
|      | |  └─ Listing.java
|      | |  └─ Category.java
|      | |  └─ ListingImage.java
|      | |  └─ ListingStatus.java
|      | |
|      | └─ valueobject
|      | |  └─ Price.java
|      | |  └─ ListingTitle.java
```

```
|      | | | | └─ ListingDescription.java
|      | | | |
|      | | | └─ repository
|      | | |   └─ ListingRepository.java
|      | | |     └─ CategoryRepository.java
|      | | |
|      | | └─ service
|      | |   └─ RecommendationService.java
|      | |     └─ ContentModerationService.java
|      | |
|      | └─ application
|      |   └─ usecase
|      |     └─ CreateListingUseCase.java
|      |       └─ UpdateListingUseCase.java
|      |         └─ BrowseListingsUseCase.java
|      |
|      | └─ dto
|      |
|      | └─ infrastructure
|      |   └─ persistence
|      |     └─ mapper
|      |
|      | └─ interfaces
|      |   └─ rest
|      |     └─ ListingController.java
|
|
|      └─ tutoring
|
|      └─ domain
|      |   └─ model
|      |     └─ TutorProfile.java
|      |       └─ TutoringSession.java
```



```

|      |  └─ service
|      |    └─ ReputationCalculationService.java
|      |    └─ FraudDetectionService.java
|      |
|      └─ application
|      └─ infrastructure
|      └─ interfaces
|
└─ resources
    |
    └─ static
        | └─ css
        | └─ js
        └─ images
    |
    └─ templates
        | └─ login.html
        | └─ register.html
        | └─ home.html
        | └─ create-listing.html
        └─ tutoring.html
    |
    └─ application.yml

```

15. Ventajas de Usar DDD

Modelado Claro del Negocio: El sistema refleja procesos reales del negocio.

Mejor Mantenibilidad: Cada dominio está aislado.

Menor Acoplamiento: Los contextos se comunican mediante límites controlados.

Mejor Escalabilidad: Los contextos pueden evolucionar independientemente.

Lenguaje Compartido: Los equipos técnicos y de negocio utilizan la misma terminología.

16. Desventajas de Usar DDD

Mayor Complejidad Inicial: Requiere análisis profundo del dominio.

Más Tiempo de Diseño: El modelado estratégico toma tiempo.

Curva de Aprendizaje: Los desarrolladores deben comprender conceptos DDD.

Posible Sobreingeniería: Los sistemas pequeños pueden no requerir DDD completo.